



RAJALAKSHMI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

ACADEMIC YEAR 2025 - 2026

SEMESTER IV

AL23421 - MACHINE LEARNING LABORATORY

MINI PROJECT REPORT

REGISTER NUMBER	2117240030028
NAME	DHARRINI SELVI R
PROJECT TITLE	SPAM EMAIL CLASSIFICATION USING NAÏVE BAYES AND LOGISTIC REGRESSION
DATE OF SUBMISSION	
FACULTY IN-CHARGE	Mrs. M. Divya

Signature of Faculty In-charge

Signature of HoD

Internal Examiner

External Examiner

INTRODUCTION

Machine Learning is a subset of Artificial Intelligence that enables systems to learn patterns from data and make predictions without being explicitly programmed. In the domain of communication systems, machine learning plays a vital role in automatically identifying and filtering unwanted messages, thereby improving user experience and ensuring better security.

In this project, supervised learning algorithms such as Naive Bayes and Logistic Regression are used to classify messages as spam or ham (non-spam). These models are trained on a dataset of labeled messages and learn to recognize patterns in textual data. Based on these learned patterns, the system can determine whether a new message is legitimate or potentially harmful.

Spam messages have become a significant issue in digital communication, often leading to serious problems such as phishing attacks, fraud, and data breaches. Efficient spam detection is essential to protect users from malicious content and to maintain the reliability of communication platforms. This project aims to develop an effective spam detection system with a user-friendly graphical interface using Tkinter, allowing users to input messages, obtain classification results, and compare the performance of both machine learning models.

PROBLEM STATEMENT

To develop a machine learning model that classifies messages as **spam** or **ham (non-spam)** based on textual content using algorithms such as **Naive Bayes** and **Logistic Regression**, and to provide an interactive graphical interface for real-time message classification.

GOAL

The primary goal of this project is to achieve accurate classification of messages into Spam and Ham (Non-Spam) categories using machine learning algorithms such as Naive Bayes and Logistic Regression. By training these models on labeled data, the system aims to effectively distinguish between legitimate and unwanted messages.

Another important objective is to provide a reliable and efficient system that can automatically detect spam messages based on their textual content. This helps in reducing manual effort and enhances the security of communication by identifying potentially harmful messages.

The project also focuses on developing a simple and user-friendly graphical interface (GUI) using Tkinter, which enables users to input messages and obtain real-time classification results. This makes the system accessible and easy to use even for non-technical users.

Furthermore, the project explores the possibility of extending the system for real-world applications such as email filtering, messaging platforms, and cybersecurity systems, making it practical and scalable for future use.

THEORETICAL BACKGROUND

Overview of the Problem

Spam message classification is a **text classification problem** in Machine Learning, where the goal is to categorize messages into two classes: **Spam (1)** and **Ham (0)**. This is a supervised learning task, where models are trained using labeled datasets containing examples of both spam and non-spam messages. The challenge lies in accurately identifying patterns in textual data and generalizing them to unseen messages. To process textual data, it must be converted into numerical form. This is achieved using techniques like **TF-IDF (Term Frequency–Inverse Document Frequency)**, which represents the importance of words in a document relative to the entire dataset.

TF-IDF Representation

TF-IDF helps in transforming text into feature vectors:

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$
$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$
$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Where:

- t = term (word)
- d = document (message)
- N = total number of documents
- $df(t)$ = number of documents containing term t

Naive Bayes Algorithm

Naive Bayes is a probabilistic classifier based on **Bayes' Theorem**, assuming independence between features.

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

Where:

- C = class (Spam or Ham)
- X = feature vector (TF-IDF values)

For classification:

$$\hat{C} = \arg \max_C P(C) \prod_{i=1}^n P(x_i | C)$$

Naive Bayes is efficient and works well for text data due to its simplicity and speed.

Logistic Regression

Logistic Regression is a statistical model used for binary classification. It predicts the probability of a class using the **sigmoid function**:

$$P(y = 1 | X) = \frac{1}{1 + e^{-(w^T X + b)}}$$

Where:

- w = weight vector
- X = input features
- b = bias

The output is a probability between 0 and 1, which is then thresholded to classify the message as spam or ham.

Literature Survey

Several studies have explored spam detection using machine learning techniques to improve the accuracy and efficiency of message classification systems. Traditional methods such as Naive Bayes have been widely used due to their simplicity, speed, and effectiveness in handling high-dimensional text data. These models perform well even with limited computational resources and are particularly suitable for basic text classification tasks.

Logistic Regression has also been applied in spam detection and has shown improved performance in scenarios where relationships between features are more complex. It provides better predictive capability by modeling probabilities and handling feature interactions more effectively than simpler models.

In addition to these methods, other algorithms such as Support Vector Machines (SVM), Decision Trees, and Random Forests have been used for spam classification, each offering different advantages in terms of accuracy and computational complexity. Overall, these studies indicate that while advanced models offer improved performance, simpler algorithms like Naive Bayes remain competitive for text classification tasks due to their efficiency and lower computational cost.

Justification for Choosing the Algorithms

Naive Bayes is chosen for this project because it is fast, efficient, and highly suitable for text classification problems. It performs well even with limited training data and is capable of handling high-dimensional feature spaces effectively, making it an ideal choice for spam detection tasks involving textual data.

Logistic Regression is selected due to its ability to model more complex relationships between features and provide improved accuracy in certain scenarios. It also produces probability-based outputs, which help in better interpretation and understanding of the model's predictions.

By implementing and comparing both algorithms, the project enables evaluation of the trade-offs between speed and accuracy. This comparison provides a deeper understanding of their performance and effectiveness in detecting spam messages, helping to identify the most suitable approach for the problem.

ALGORITHM EXPLANATION WITH EXAMPLE

ALGORITHM

1. Input Data

Message dataset is given as input:

- Text Message (SMS/Email)
- Label:
 - Spam (1)
 - Ham (0)

2. Text Preprocessing

The input text is cleaned and prepared:

- Convert text to lowercase
- Remove punctuation and special characters
- Remove unnecessary spaces

3. Feature Extraction (TF-IDF)

Text data is converted into numerical form:

- Each message is transformed into a vector
- Important words get higher weights

Mathematical Representation:

$$TF(t, d) = \frac{\text{Frequency of term } t}{\text{Total terms in document}}$$
$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$
$$TF-IDF = TF \times IDF$$

4. Model Training

Two models are trained using the processed data:

Naive Bayes

- Calculates probability of each class
- Assumes features are independent

Logistic Regression

- Learns relationship between features and output
- Uses weights and bias for prediction

5. Prediction

For a new input message:

- Convert message into TF-IDF vector
- Apply selected model

Naive Bayes:

$$P(C | X) = P(C) \prod P(x_i | C)$$

Logistic Regression:

$$P(y = 1) = \frac{1}{1 + e^{-(w^T X + b)}}$$

6. Output Classification

Final output is predicted as:

- Spam
- Ham

Based on:

- Highest probability (Naive Bayes)
- Threshold (Logistic Regression, usually 0.5)

Mathematical Representation

Naive Bayes:

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

Logistic Regression (Sigmoid Function):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sample Data

Message	Class
"Win money now"	Spam
"Call me tonight"	Ham
"Free prize available"	Spam
"Let's meet tomorrow"	Ham

NAÏVE BAYES EXPLANATION:

Working Example

Consider a new message:

"Win free money"

Step 1: Calculate prior probabilities

- $P(\text{Spam}) = 2/4 = 0.5$
- $P(\text{Ham}) = 2/4 = 0.5$

Step 2: Likelihood estimation (simplified)

- Words like *win*, *free*, *money* occur more frequently in spam messages

Step 3: Compute posterior probability

$$P(\text{Spam} | X) > P(\text{Ham} | X)$$

Therefore, the message is classified as **Spam**.

LOGISTIC REGRESSION EXPLANATION:

Working Example

For the message "**Win free money**", assume:

$$z = w^T X + b = 2.3$$
$$P = \frac{1}{1 + e^{-2.3}} \approx 0.91$$

Since $P > 0.5$, the message is classified as **Spam**

Sample Outcome Representation

Confusion Matrix (Example)

Actual / Predicted	Spam	Ham
Spam	45	5
Ham	3	47

- True Positives (TP) = 45
- True Negatives (TN) = 47
- False Positives (FP) = 3
- False Negatives (FN) = 5

Performance Metrics (Example)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{45 + 47}{100} = 0.92$$

$$Precision = \frac{TP}{TP + FP} = \frac{45}{45 + 3} = 0.94$$

$$Recall = \frac{TP}{TP + FN} = \frac{45}{45 + 5} = 0.90$$

$$F1\ Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \approx 0.92$$

Graphical Representation (Explanation)

The Line or Bar Graph is used to compare important performance metrics such as Accuracy, Precision, Recall, and F1 Score for both Naive Bayes and Logistic Regression algorithms. This graphical representation provides a clear and intuitive understanding of how each model performs across different evaluation criteria, making it easier to identify strengths and weaknesses at a glance.

The Confusion Matrix Heatmap is another essential visualization that represents the number of correct and incorrect predictions made by the models. It clearly shows values such as true positives, true negatives, false positives, and false negatives, allowing for a deeper analysis of classification performance. This helps in understanding how well the model distinguishes between spam and ham messages.

Additionally, the Model Comparison Chart provides a side-by-side comparison of both algorithms based on their evaluation metrics. This visualization helps in identifying which algorithm performs better overall and highlights the trade-offs between them, such as accuracy versus recall. Together, these graphical representations enhance the interpretability of the results and support better decision-making regarding model selection.

IMPLEMENTATION CODE AND DATASET USED

Dataset Used – <https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>

Category	Description / Example
Dataset Name	SMS Spam Collection Dataset
Domain	Natural Language Processing (NLP)
Objective	Classification
Data Source	Public (Kaggle)
Number of Samples	5,572 messages
Data Type	Text
Instance Description	Each sample represents a single SMS message labeled as spam or ham
Features (Inputs)	Message text content
Feature Types	Text
Target Variable	Label (Spam = 1, Ham = 0)
Number of Classes	2 (Spam, Ham)
License	Public dataset (UCI / Kaggle usage)

PYTHON CODE

Spam_classification.py

```
import tkinter as tk

from tkinter import messagebox

import pandas as pd

import string

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.linear_model import LogisticRegression


# Load dataset

df = pd.read_csv("spam.csv", encoding='latin-1')

df = df[['v1', 'v2']]

df.columns = ['label', 'message']

df['label'] = df['label'].map({'ham': 0, 'spam': 1})


# Text cleaning

def clean_text(text):

    text = text.lower()

    return "".join([char for char in text if char not in string.punctuation])


df['message'] = df['message'].apply(clean_text)
```

```
# Split data

X_train, X_test, y_train, y_test = train_test_split(
    df['message'], df['label'], test_size=0.2, random_state=42
)

# Vectorization

vectorizer = TfidfVectorizer()

X_train_vec = vectorizer.fit_transform(X_train)

X_test_vec = vectorizer.transform(X_test)

# Models

nb_model = MultinomialNB()

lr_model = LogisticRegression(max_iter=1000)

nb_model.fit(X_train_vec, y_train)

lr_model.fit(X_train_vec, y_train)

# Prediction

def predict_message():

    message = text_entry.get("1.0", tk.END).strip()

    if message == "":

        messagebox.showwarning("Warning", "Please enter a message!")
```

<Spam Email Classification using Naïve Bayes and Logistic Regression >

```
    return

cleaned = clean_text(message)

vectorized = vectorizer.transform([cleaned])

if model_var.get() == "Naive Bayes":

    prediction = nb_model.predict(vectorized)[0]

else:

    prediction = lr_model.predict(vectorized)[0]

result_label.config(

    text="SPAM ❌ " if prediction == 1 else "HAM ✅ ",

    fg="red" if prediction == 1 else "green"

)


# GUI

root = tk.Tk()

root.title("Spam Email Classifier")

root.geometry("600x500")

root.config(bg="#1e1e2f")


tk.Label(root, text="Spam Email Classifier",

        font=("Helvetica", 20, "bold"),

        bg="#1e1e2f", fg="white").pack(pady=20)


text_entry = tk.Text(root, height=8, width=50, font=("Arial", 12))
```

<Spam Email Classification using Naïve Bayes and Logistic Regression >

```
text_entry.pack(pady=10)
```

```
model_var = tk.StringVar(value="Naive Bayes")
```

```
frame = tk.Frame(root, bg="#1e1e2f")
```

```
frame.pack(pady=10)
```

```
tk.Radiobutton(frame, text="Naive Bayes", variable=model_var,  
               value="Naive Bayes", bg="#1e1e2f", fg="white").grid(row=0, column=0, padx=20)
```

```
tk.Radiobutton(frame, text="Logistic Regression", variable=model_var,  
               value="Logistic Regression", bg="#1e1e2f", fg="white").grid(row=0, column=1,  
padx=20)
```

```
tk.Button(root, text="Classify Message",  
          command=predict_message,  
          font=("Arial", 14, "bold"),  
          bg="#4CAF50", fg="white").pack(pady=20)
```

```
result_label = tk.Label(root, text="", font=("Arial", 16, "bold"), bg="#1e1e2f")
```

```
result_label.pack(pady=10)
```

```
root.mainloop()
```

```
# Evaluation
```

<Spam Email Classification using Naïve Bayes and Logistic Regression >

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,  
confusion_matrix
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
print("\n 🚦 Running Model Evaluation...\n")
```

```
nb_preds = nb_model.predict(X_test_vec)
```

```
lr_preds = lr_model.predict(X_test_vec)
```

```
def evaluate_model(name, y_true, y_pred):
```

```
    acc = accuracy_score(y_true, y_pred)
```

```
    prec = precision_score(y_true, y_pred)
```

```
    rec = recall_score(y_true, y_pred)
```

```
    f1 = f1_score(y_true, y_pred)
```

```
    print(f'{name}')
```

```
    print(f'Accuracy: {acc:.4f}, Precision: {prec:.4f}, Recall: {rec:.4f}, F1: {f1:.4f}\n')
```

```
    return acc, prec, rec, f1
```

```
nb_metrics = evaluate_model("Naive Bayes", y_test, nb_preds)
```

```
lr_metrics = evaluate_model("Logistic Regression", y_test, lr_preds)
```

```
def plot_confusion(y_true, y_pred, title):  
    cm = confusion_matrix(y_true, y_pred)  
    plt.figure()  
    sns.heatmap(cm, annot=True, fmt='d')  
    plt.title(title)  
    plt.xlabel("Predicted")  
    plt.ylabel("Actual")  
    plt.show()  
  
plot_confusion(y_test, nb_preds, "Naive Bayes Confusion Matrix")  
plot_confusion(y_test, lr_preds, "Logistic Regression Confusion Matrix")  
labels = ['Accuracy', 'Precision', 'Recall', 'F1 Score']  
x = range(len(labels))  
plt.figure()  
plt.plot(x, list(nb_metrics), marker='o', label='Naive Bayes')  
plt.plot(x, list(lr_metrics), marker='o', label='Logistic Regression')  
plt.xticks(x, labels)  
plt.title("Model Performance Comparison")  
plt.legend()  
plt.grid()  
plt.show()
```

OUTPUT

Ham Message Detection

1. Using Naive Bayes

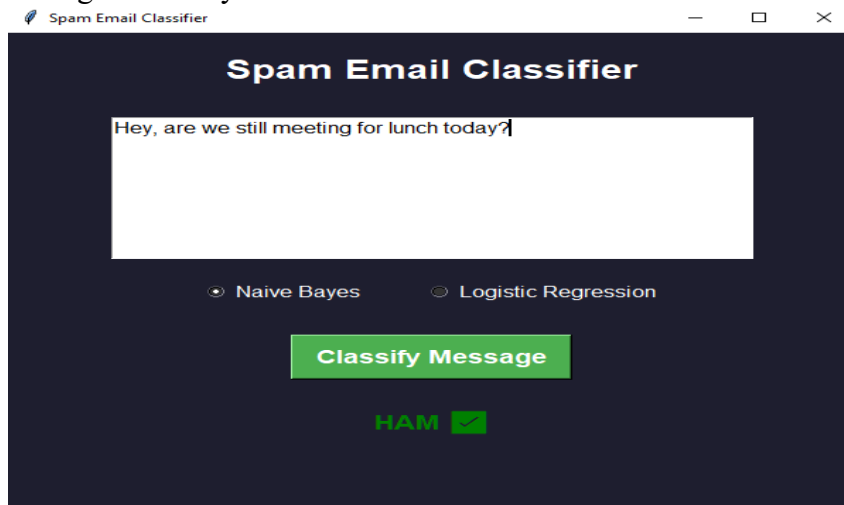


FIGURE 1: Ham Message Classification

The model correctly identifies the message as a normal (non-spam) message based on conversational words like *meeting* and *lunch*.

2. Using Logistic Regression

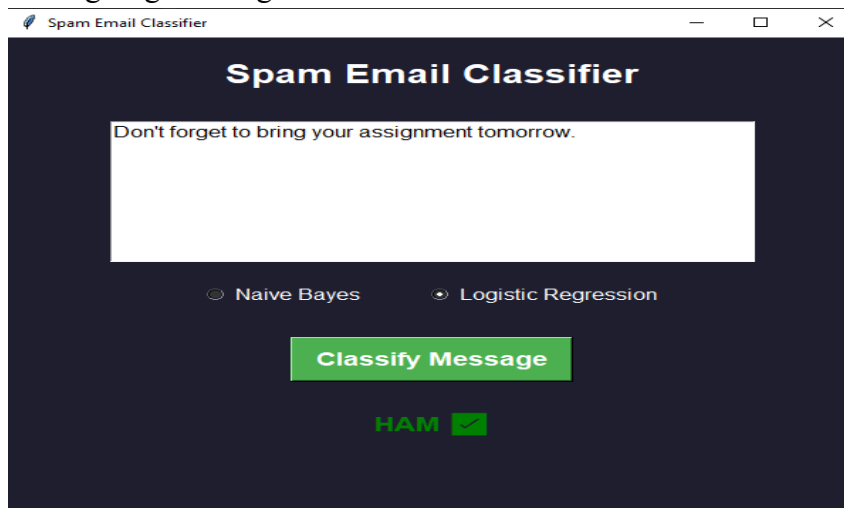


FIGURE 2: Ham Message Classification

The message contains academic context and no spam indicators, so it is correctly classified as ham.

Spam Message Detection

1.Using Naive Bayes

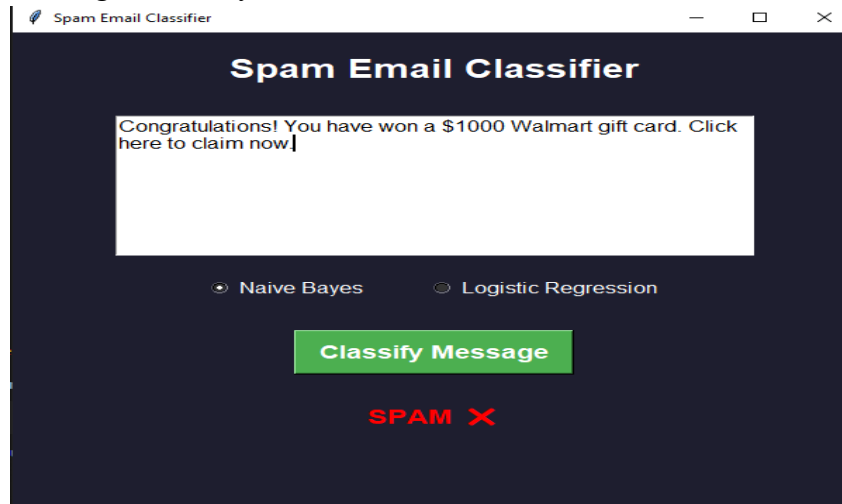


FIGURE 3: Spam Message Classification

The message contains promotional keywords like “won,” “gift card,” and “claim now,” so it is correctly classified as spam.

2.Using Logistic Regression

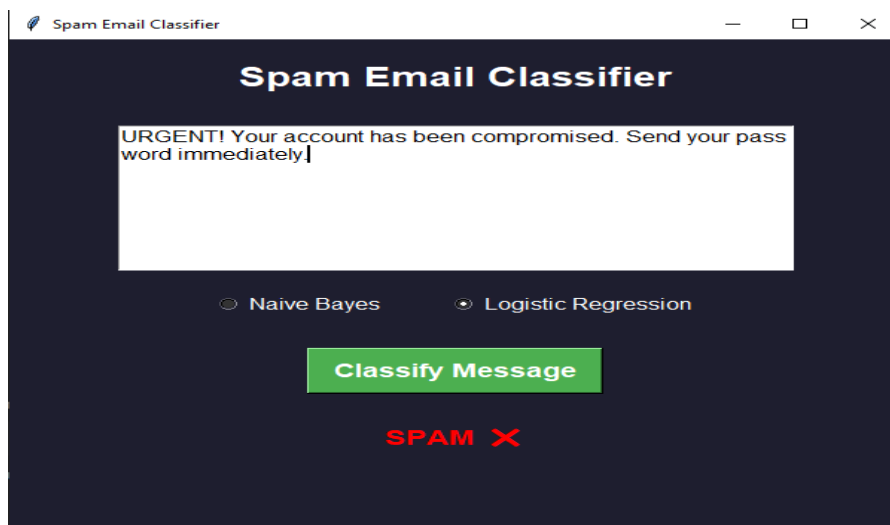


FIGURE 4: Spam Message Classification

The message contains urgent and sensitive keywords like “URGENT” and “send your password,” so it is correctly classified as spam.

CONFUSION MATRIX ANALYSIS

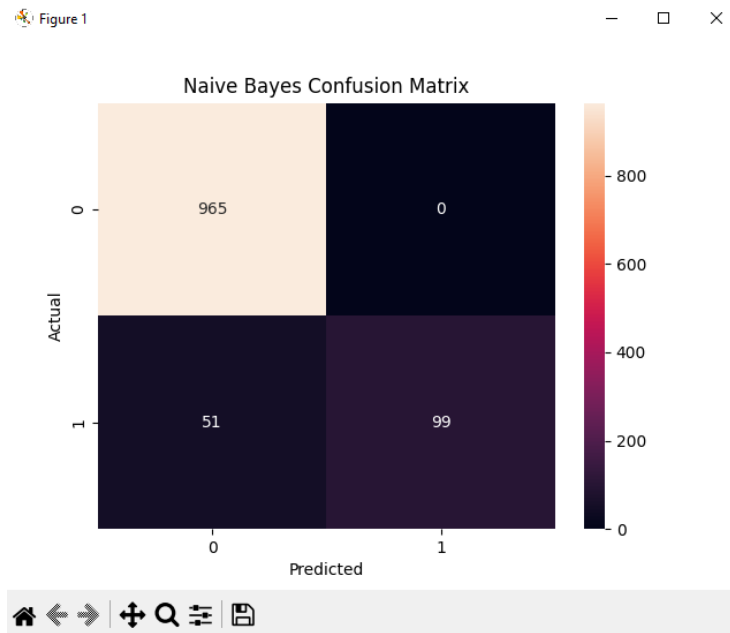


FIGURE 5: Naïve Bayes Confusion Matrix

Naïve Bayes has **perfect precision (1.0)** but lower recall, meaning it misses some spam messages.

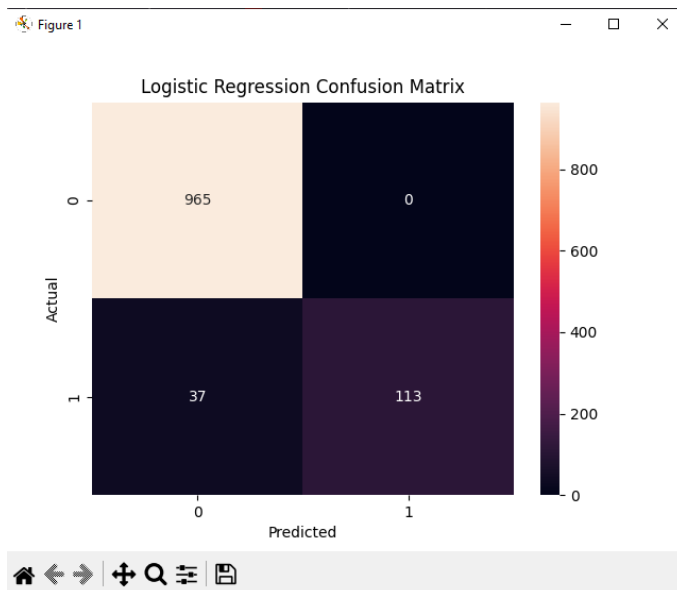


FIGURE 6: Logistic Regression Confusion Matrix

Logistic Regression detects more spam messages (higher recall), making it more reliable overall.

PERFORMANCE METRICS

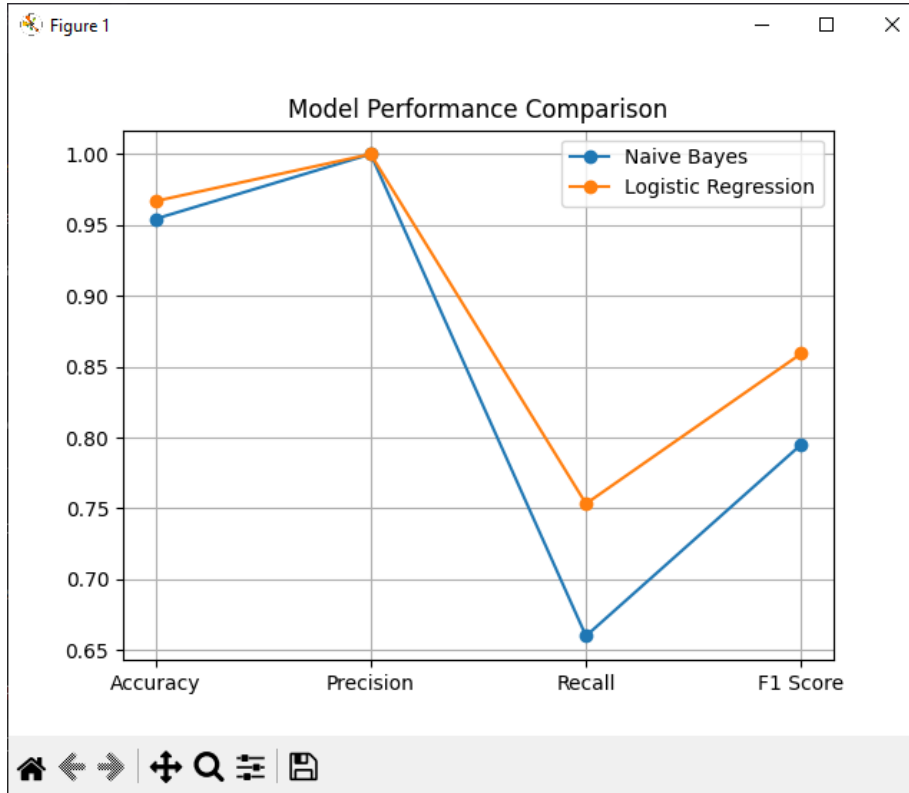


FIGURE 7: Performance Metrics

```
Running Model Evaluation...

♦ Naive Bayes Performance:

Accuracy : 0.9543
Precision : 1.0000
Recall   : 0.6600
F1 Score : 0.7952
-----

♦ Logistic Regression Performance:

Accuracy : 0.9668
Precision : 1.0000
Recall   : 0.7533
F1 Score : 0.8593
-----
```

FIGURE 8: Model Evaluation

RESULTS AND FUTURE ENHANCEMENT

Results

The developed spam classification system successfully identifies messages as Spam or Ham using machine learning techniques. Both Naive Bayes and Logistic Regression models were implemented and evaluated on the dataset to analyse their performance in text classification.

The system achieved high accuracy, with Logistic Regression (96.68%) slightly outperforming Naive Bayes (95.43%). Both models demonstrated perfect precision (1.0), which indicates that no legitimate messages were incorrectly classified as spam. However, Logistic Regression showed higher recall (0.7533) compared to Naive Bayes (0.6600), meaning it was more effective in detecting actual spam messages. The F1 Score further confirms that Logistic Regression provides a more balanced performance between precision and recall.

Overall, the system performs efficiently and delivers reliable results. The integration of a user-friendly graphical interface (GUI) enables real-time message classification, making the system practical and easy to use.

Comparison with Other Approaches

Traditional filtering methods such as keyword-based filtering are less effective in spam detection, as they fail to adapt to evolving spam patterns and can be easily bypassed. Machine learning models like Naive Bayes are widely used due to their speed and suitability for text classification tasks; however, they may miss more complex patterns present in spam messages.

Logistic Regression improves prediction accuracy by learning relationships between features and provides better generalization in certain cases. More advanced algorithms such as Support Vector Machines (SVM) and Random Forests can achieve higher accuracy, but they require greater computational resources and longer training time.

Compared to these approaches, the proposed system achieves a good balance between accuracy, efficiency, and simplicity, making it suitable for practical applications such as real-time spam detection.

Future Enhancements

The current spam classification system can be further improved by integrating more advanced algorithms such as Support Vector Machines (SVM) or deep learning models to achieve higher accuracy and better performance. Expanding the dataset to include emails, multilingual messages, and real-time data can enhance the model's ability to generalize across different types of communication.

The graphical user interface (GUI) can also be enhanced by adding features such as message history, confidence score display, and batch processing for improved usability.

Additionally, the system can be deployed as a web or mobile application to make it accessible for real-world usage.

Future improvements may also include implementing continuous learning mechanisms to adapt to evolving spam patterns over time. Incorporating additional features such as sender information, embedded links, and attachments can further strengthen the model's ability to accurately classify messages.

Git Hub Link of the project along with the report	https://github.com/dharrini06/Spam-Email-Classification-using-Naive-Bayes-and-Logistic-Regression
--	--

REFERENCES

1. UCI Machine Learning Repository, “*SMS Spam Collection Dataset*”. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/sms+spam+collection>
2. Kaggle, “*SMS Spam Collection Dataset*”. [Online]. Available: <https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>
3. T. M. Mitchell, *Machine Learning*, McGraw-Hill Education, 1997.
4. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
5. S. Raschka and V. Mirjalili, *Python Machine Learning*, 3rd Edition, Packt Publishing, 2019.
6. Scikit-learn Documentation, “*Machine Learning in Python*”. [Online]. Available: <https://scikit-learn.org/stable/>